

1. 애플리케이션 아키텍처 소개

❖ 1단계 : 단일 애플리케이션

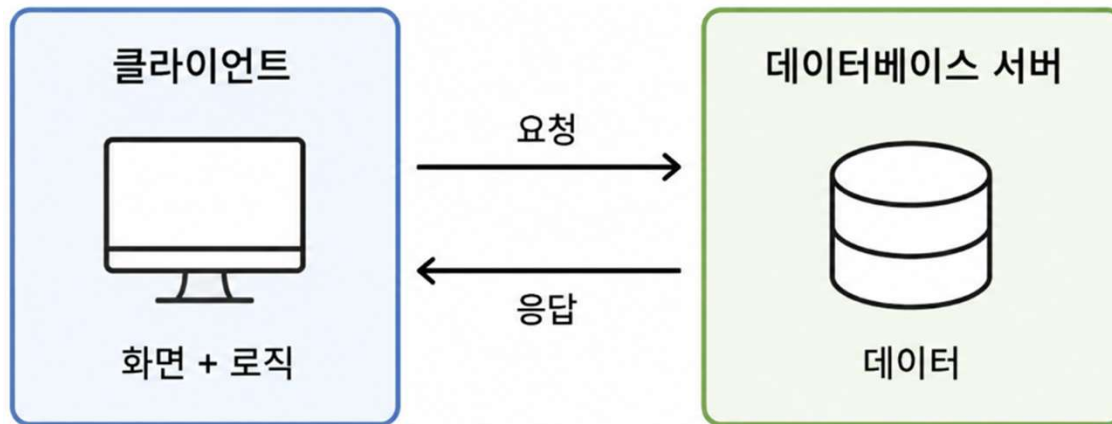
- 다른 외부요소와 통신하지 않음. 모든 요소가 한 컴퓨터 안에서 실행됨
- 예시 : 윈도우에서 Notepad를 사용하는 경우
- 단점
 - 여러 사람이 동시에 이용하는 서비스로 확장할 수 없음



1. 애플리케이션 아키텍처 소개

❖ 2단계-1 : 2-tier

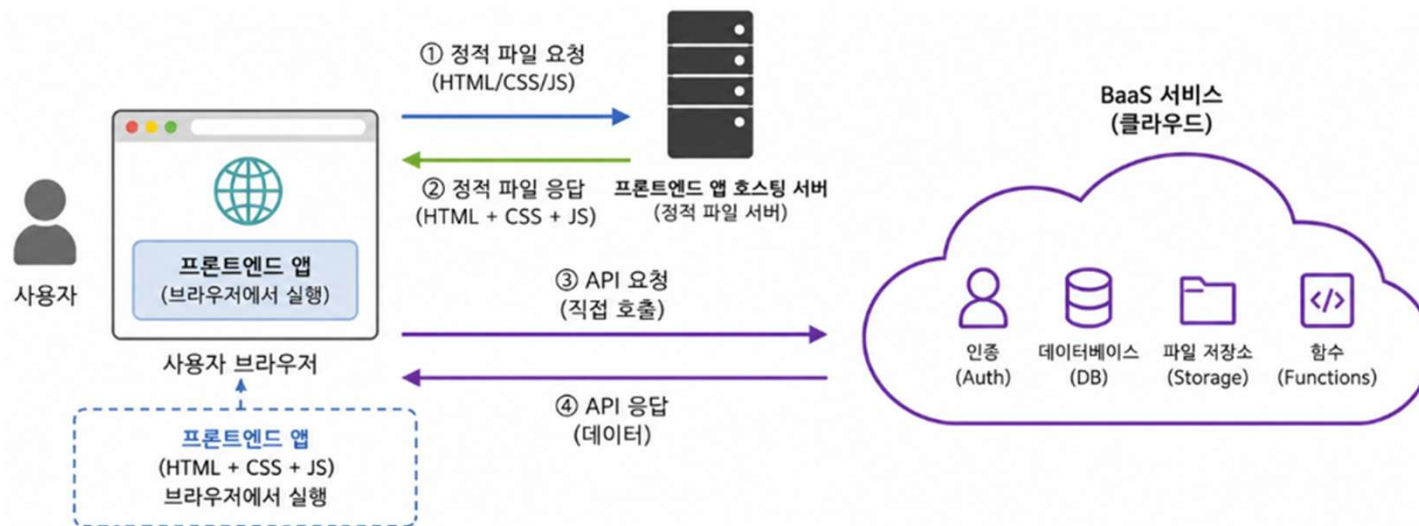
- 화면 기능, 비즈니스 로직 기능이 한 컴퓨터에서 실행되고, 데이터를 별도 서버에 저장함
- 예시 : 회사 사무실의 오래된 그룹웨어 시스템-직원마다 프로그램을 설치하고 회사 DB 에 접속
- 단점
 - 비즈니스 로직을 수정하면 애플리케이션을 사용자 PC마다 재배포해야 함



1. 애플리케이션 아키텍처 소개

❖ 2단계-2 : 2-tier + 웹 확장

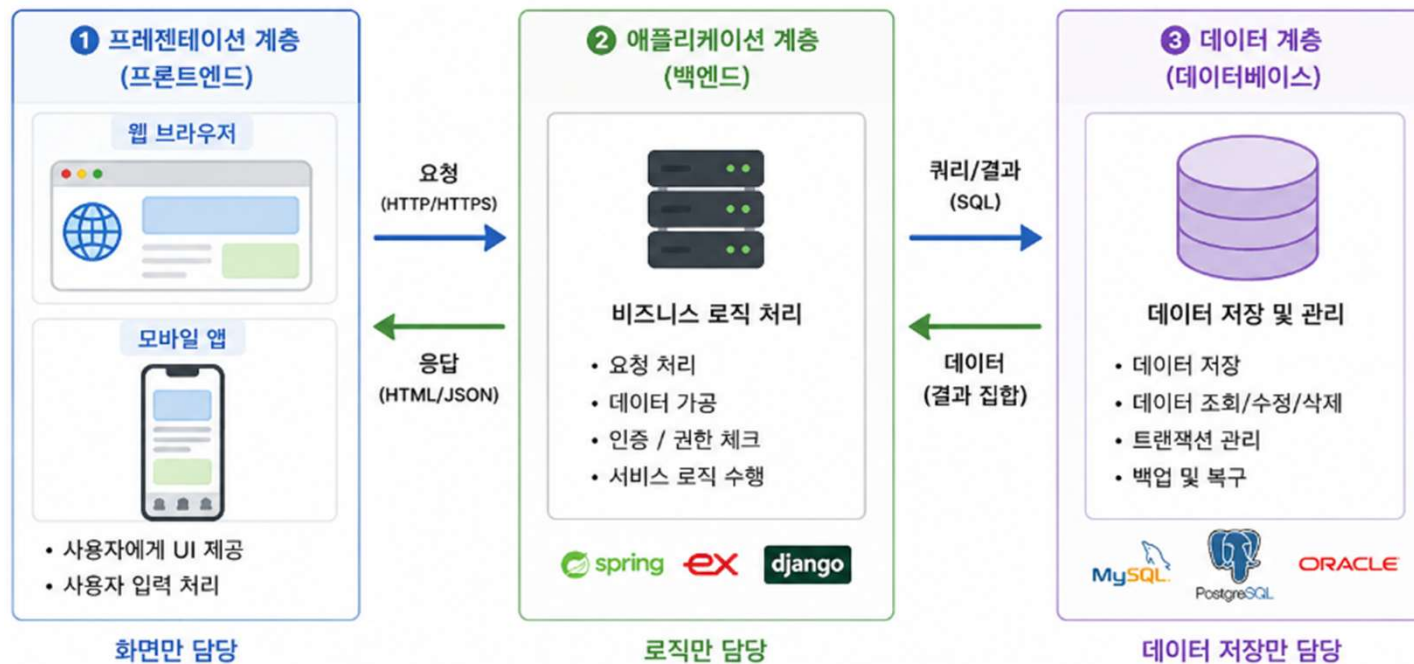
- 화면 기능과 간단한 로직만 프론트엔드 애플리케이션으로 작성
- 예시 : React 앱 + Supabase 또는 Vue.js 앱 + Google Firebase
- 단점
 - 브라우저에서 실행되는 앱은 데이터베이스에 직접 접근할 수 없음(Sandbox 보안 모델)
 - 자체적인 백엔드가 있어야 함
 - 자체적인 백엔드 API를 사용하지 않으려면 반드시 BaaS(Backend as a Service : 예-supabase) 를 이용해야 함



1. 애플리케이션 아키텍처 소개

❖ 3단계 : 3-tier

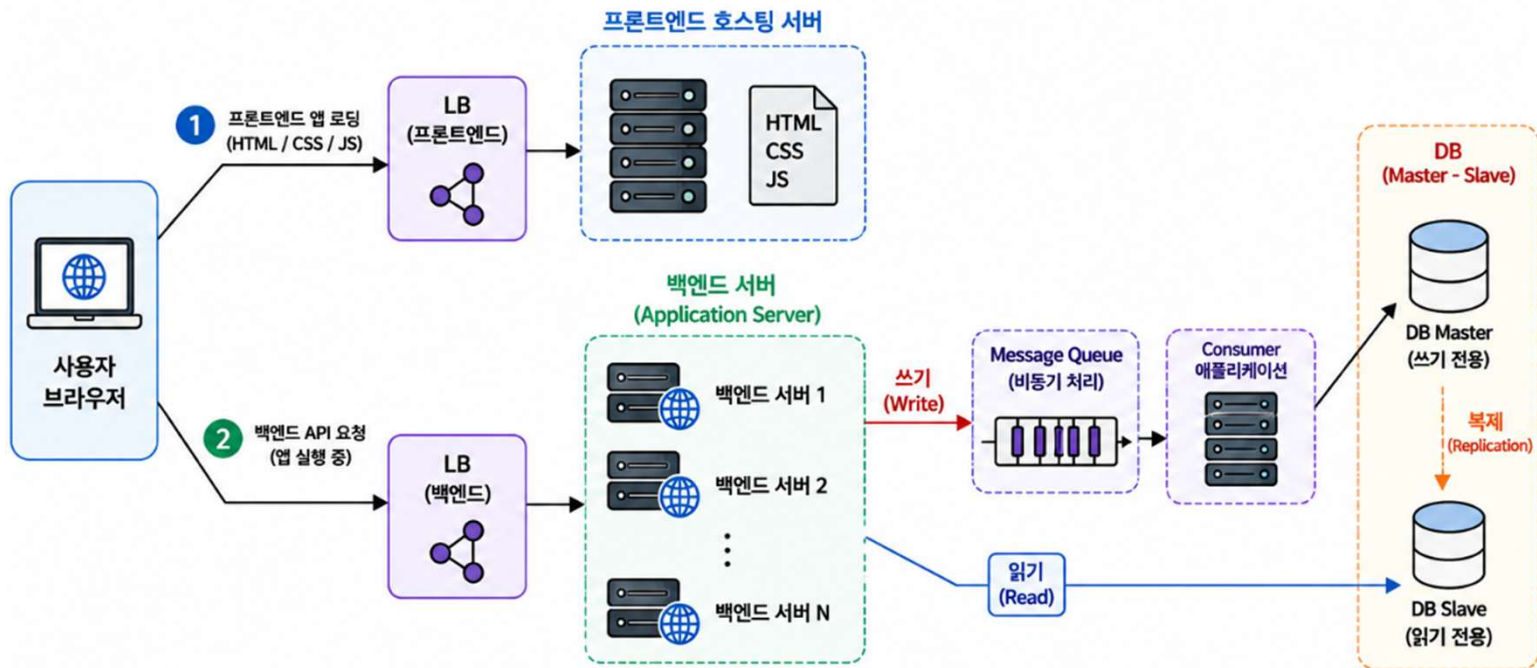
- 화면 기능, 비즈니스 로직, 데이터베이스 계층을 분리된 컴퓨터에서 실행함
- 비즈니스 로직이 변경되더라도 프리젠테이션 계층의 앱은 변경하지 않아도 됨
- 계층의 분리는 되었지만고가용성(High Availability) 구성이 된 것은 아님



1. 애플리케이션 아키텍처 소개

❖ 4단계 : N-tier

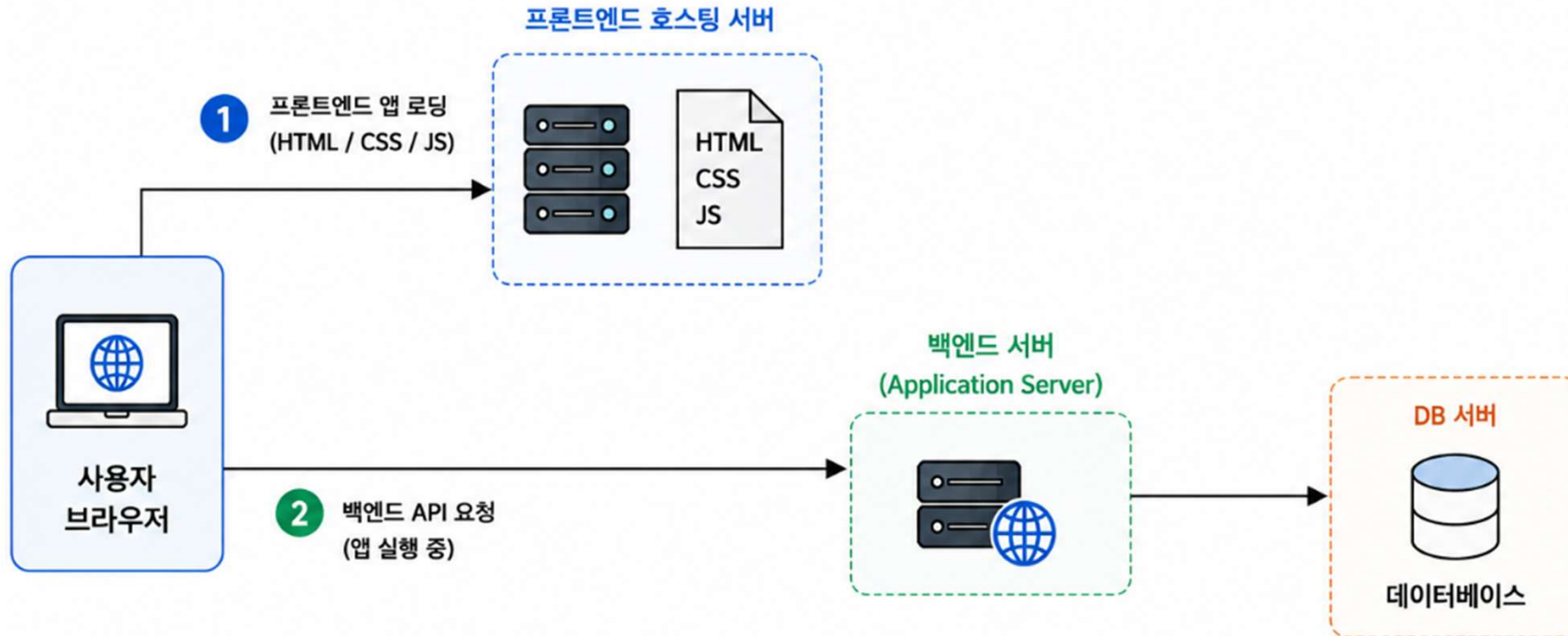
- 고가용성(High Availability)을 위한 다양한 계층이 추가됨
 - Load Balancer(LB) : 부하분산, 고가용성 구성
 - Message Queue : 서버 요청 수요 폭증을 대비한 비동기 처리
 - DB 확장 : DB복제(Replication), 샤딩(Sharding) 등의 기법을 적용



1. 애플리케이션 아키텍처 소개

❖이 과정에서 적용할 아키텍처 : 3-tier with web

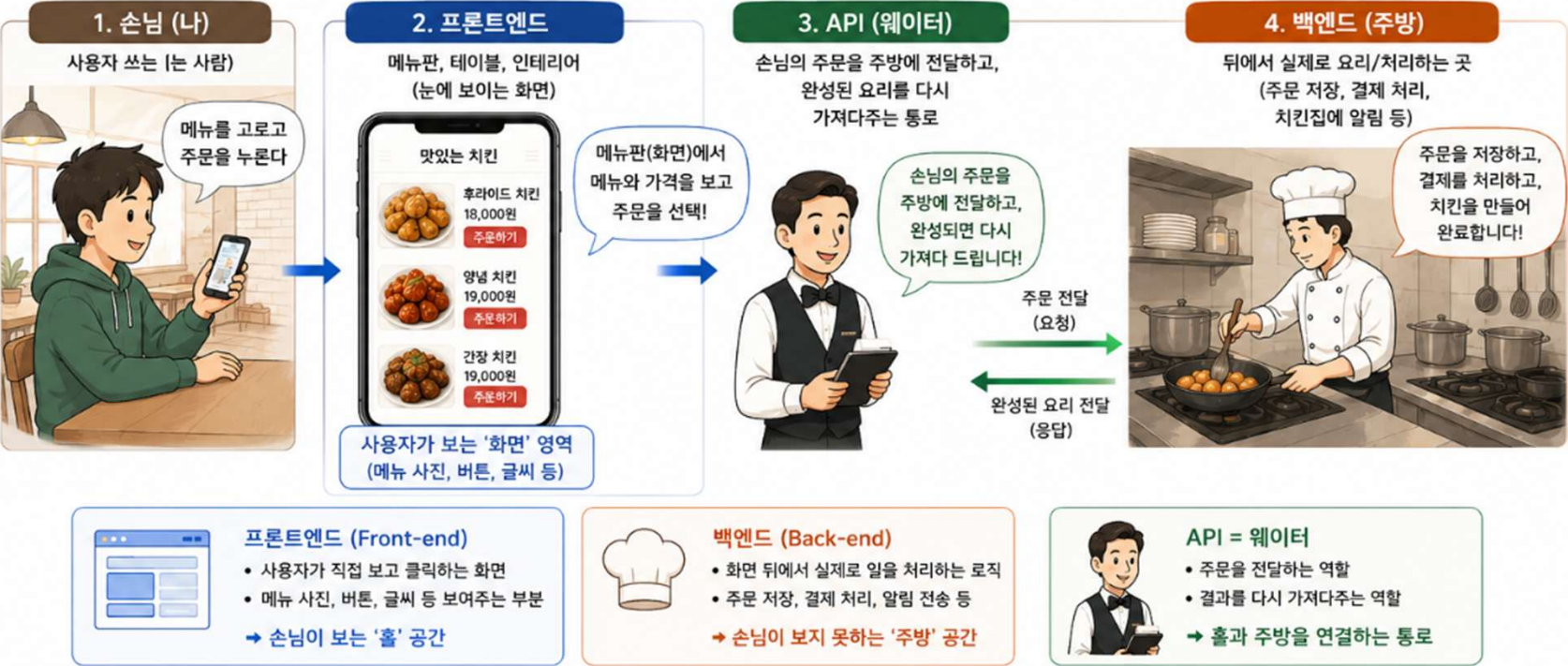
- 프론트엔드앱 + 백엔드API + DB
 - 가장 많이 적용되는 애플리케이션 배치 아키텍처



2. 프론트엔드와 백엔드의 처리 과정

레스토랑의 업무 처리 과정으로 비유

레스토랑	 손님 (나)	 메뉴판, 테이블, 인테리어	 웨이터	 주방
IT 서비스	사용자 (앱을 쓰는 사람)	프론트엔드 (눈에 보이는 화면)	API (손님의 주문을 주방에 전달하고, 완성된 요리를 다시 가져다주는 통로)	백엔드 (뒤에서 실제로 요리/처리하는 곳)



3. 백엔드 개발 언어와 프레임워크

대표 백엔드 언어 & 프레임워크

주방(백엔드)에서 요리(데이터 처리)를 하기 위한 언어와 도구들

JavaScript (Node.js)	Python	Java
 Node.js	 Python	 Java
대표 프레임워크 Express, NestJS	대표 프레임워크 Django, FastAPI	대표 프레임워크 Spring / Spring Boot
비유  프론트엔드도 만들고 백엔드도 만드는 “만능 요리사” 한 가지 언어로 화면과 서버를 모두 다룰 수 있어 입문자에게 인기!	비유  “레시피가 잘 정리된 요리책” 문법이 쉽고 읽기 편해서 초보자도 빠르게 배움	비유  “대형 프랜차이즈 주방 시스템” 크고 안정적인 서비스 (은행, 대기업)에서 많이 사용
특징  • 웹 전체를 JavaScript 하나로 개발 가능 • 학습 진입 장벽이 낮고 커뮤니티가 큼 • 빠르게 개발하고 배포하기 쉬움	특징  • 문법이 간결하고 이해하기 쉬움 • 다양한 기능이 프레임워크에 잘 정리됨 • 데이터 분석, AI 등 다양한 분야에도 활용	특징  • 성능과 안정성이 뛰어남 • 대규모 시스템 구축에 적합 • 기업 환경에서 표준으로 널리 사용

★ 핵심은 언어 자체가 아니라 개념!

같은 김치찌개도 어떤 주방장은 가스레인지로, 어떤 주방장은 인덕션으로 끓이지만 결과물(요리)은 같은 것처럼, 언어가 달라도 결국 하는 일(데이터 처리)은 비슷합니다.



4. API

API (Application Programming Interface) = “주방과 손님 사이의 주문 규칙”



대표적인 API 방식 비교

1. RESTful API

비유: 정해진 메뉴판에서 번호로 주문
("1번 메뉴 주세요")



특징

- 가장 널리 쓰이는 방식
- URL(주소)과 동적(GET/POST 등)으로 요청을 구분
- 간단하고 이해하기 쉬움

예시

GET /api/menu/1
→ 김치찌개 정보 출력
POST /api/order
→ 주문 생성

2. GraphQL

비유: 필요한 재료만 골라서
커스텀 주문
("이 요리에서 이 재료만 빼고,
이 재료는 추가해서 딱 필요한 만큼만 주세요")



특징

- 필요한 데이터만 정확히 받을 수 있음
- 네트워크 효율이 좋고 유연함
- 초기 학습이 REST보다 조금 더 필요

예시 (요청 예시)

```
query {
  menu(id: 3) {
    name
    ingredients(exclude: ["양파"])
    addOns: ["치즈"]
    sauce(level: "half")
    rice(amount: "small")
  }
}
```

3. SOAP (참고)

비유: 아주 격식 있는 정찬 코스 주문서
(오래되고 무거운 방식)



특징

- XML 기반의 메시지 형식
- 보안 및 표준화가 강점
- 무겁고 복잡하여 사용이 감소 중
- 은행, 공공기관 등 레거시 시스템에서 아직도 사용

예시 (형식 예시)

```
<soap:Envelope>
  <soap:Header>...</soap:Header>
  <soap:Body>
    <GetMenuRequest>
      <MenuId>3</MenuId>
    </GetMenuRequest>
  </soap:Body>
</soap:Envelope>
```

4. API

❖ API 호출 방식 테스트

- 간단한 연락처 서비스
 - RESTful API Test : Swagger UI
 - <https://contactsvc.bmaster.kro.kr/console>
 - GraphQL API Test
 - <https://contactsvc3.bmaster.kro.kr/graphql>

이 과정에서는 RESTful API
중심으로 실습합니다.

연락처 서비스 API

2.0.0

[Base url: `contactsvc2.bmaster.kro.kr/`]

<https://contactsvc2.bmaster.kro.kr/swagger.json>

테스트용으로 만든 간단한 API입니다. 매일 새벽2시에 데이터가 초기화되어 100개의 연락처만 남습니다.

Schemes

HTTPS

연락처리스트 조회

GET

/contacts

연락처 리스트를 응답합니다.

GET

/contacts_long

1초의 지연시간 후 연락처 리스트를 응답합니다.

The image shows a GraphQL IDE interface with a query editor on the left and a response viewer on the right.

Query Editor:

```
1 query {  
2   contactsAll(pageno: 1, pagesize: 10) {  
3     pageno  
4     pagesize  
5     totalcount  
6     contacts {  
7       _id  
8       name  
9       tel  
10      address  
11      photo  
12    }  
13  }  
14 }
```

Response Viewer:

Variables Headers

1

+

Contact Service GraphQL API

```
{  
  "data": {  
    "contactsAll": {  
      "pageno": 1,  
      "pagesize": 10,  
      "totalcount": 242,  
      "contacts": [  
        {  
          "_id": "6a5874d2fb3f75950b4d48f5",  
          "name": "홍길동동",  
          "tel": "010-1111-1111",  
          "address": "서울시 중구 신당동",  
          "photo":  
            "http://contactsvc3.bmaster.kro.kr/photos/noimage"  
        },  
        {  
          "_id": "6a44b8c3fa862031c1ca19cc",  
          "name": "강감찬C",  
          "tel": "010-2222-3339",  
          "address": "서울시 영등포구",  
          "photo":  
            "http://contactsvc3.bmaster.kro.kr/photos/TM6i055PE"  
        },  
        {  
          "_id": "6a44b8c3fa862031c1ca19cc",  
          "name": "강감찬C",  
          "tel": "010-2222-3339",  
          "address": "서울시 영등포구",  
          "photo":  
            "http://contactsvc3.bmaster.kro.kr/photos/TM6i055PE"  
        }  
      ]  
    }  
  }  
}
```